

### EXPERIMENT-11: JWT authorization for API endpoints

For the above application, create authorized endpoints using JWT (JSON Web Token).

**Objective :** Secure API routes by adding authentication and token-based authorization.

**Tools :** JSON Web Token, Express middleware, authentication

**Outcome :** Protected API endpoints that require a valid JWT to access.

#### Answer:

Generate a JWT on login, return it to the client, and validate the token in middleware for protected routes.

**File Name:** jwt\_app.js

**Code:**

```
// =====  
// Experiment: JWT Authorization for API Endpoints  
// Shopping app: Login -> JWT token -> Protected API routes  
// Run: node jwt_app.js   Open: http://localhost:3000  
// No npm install needed (JWT HS256 is implemented with built-in crypto)  
// =====  
  
const http = require('http');  
const crypto = require('crypto');  
  
// ----- JWT (same logic as the jsonwebtoken library, HS256) -----  
const SECRET = 'my_super_secret_key';  
  
const b64 = x => Buffer.from(x).toString('base64url');  
  
// jwt.sign(payload, secret, { expiresIn })  
function signToken(payload, expiresInSec = 3600) {  
  const now = Math.floor(Date.now() / 1000);  
  const header = b64(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));  
  const body = b64(JSON.stringify({ ...payload, iat: now, exp: now + expiresInSec }));  
  const sig = crypto.createHmac('sha256', SECRET).update(header + '.' + body).digest('base64url');  
  return header + '.' + body + '.' + sig;    // header.payload.signature  
}  
  
// jwt.verify(token, secret) -> returns payload or throws
```

```

function verifyToken(token) {
  const parts = token.split('.');
  if (parts.length !== 3) throw new Error('Malformed token');
  const [header, body, sig] = parts;

  const expected = crypto.createHmac('sha256', SECRET).update(header + '.' +
    body).digest('base64url');
  const a = Buffer.from(sig), b = Buffer.from(expected);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) throw new Error('Invalid signature');

  if (JSON.parse(Buffer.from(header, 'base64url')).alg !== 'HS256') throw new Error('Bad algorithm');

  const payload = JSON.parse(Buffer.from(body, 'base64url'));
  if (payload.exp < Math.floor(Date.now() / 1000)) throw new Error('Token expired');
  return payload;
}

// ----- DATA (in memory) -----
const USERS = [
  { username: 'student', password: '1234', role: 'user' },
  { username: 'admin', password: 'admin123', role: 'admin' },
];
let products = [
  { id: 1, name: 'Book', price: 300 },
  { id: 2, name: 'Pen', price: 20 },
  { id: 3, name: 'Notebook', price: 80 },
  { id: 4, name: 'Bag', price: 900 },
];
const carts = {}; // username -> { productId: qty }
const history = {}; // username -> [transactions]

// ----- HELPERS -----
function send(res, status, obj) {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(obj));
}

```

```

function readJSON(req) {
  return new Promise(resolve => {
    let d = '';
    req.on('data', c => (d += c));
    req.on('end', () => { try { resolve(JSON.parse(d || '{}')); } catch { resolve({}); } });
  });
}

// ----- MIDDLEWARE -----
// Authentication: checks "Authorization: Bearer <token>"
function auth(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer '))
    return send(res, 401, { error: 'Access denied. Token missing.' });
  try {
    req.user = verifyToken(header.split(' ')[1]); // attach payload to request
    next();
  } catch (err) {
    send(res, 401, { error: 'Invalid token: ' + err.message });
  }
}

// Authorization: checks the role stored inside the token
const requireRole = role => (req, res, next) =>
  req.user.role === role ? next() : send(res, 403, { error: 'Forbidden. ' + role + ' role required.' });

// ----- ROUTE HANDLERS (controllers) -----
function login(req, res) {
  const { username, password } = req.body;
  const user = USERS.find(u => u.username === username && u.password === password);
  if (!user) return send(res, 401, { error: 'Invalid username or password' });
  const token = signToken({ username: user.username, role: user.role });
  send(res, 200, { message: 'Login successful', token });
}

```

```
const profile = (req, res) => send(res, 200, { tokenPayload: req.user });
```

```
const listProducts = (req, res) => send(res, 200, products);
```

```
function addToCart(req, res) {  
  const { productId } = req.body;  
  if (!products.find(p => p.id === productId)) return send(res, 404, { error: 'Product not found' });  
  const cart = (carts[req.user.username] || {});  
  cart[productId] = (cart[productId] || 0) + 1;  
  send(res, 200, { message: 'Added to cart', cart });  
}
```

```
function viewCart(req, res) {  
  const cart = carts[req.user.username] || {};  
  let total = 0;  
  const items = Object.entries(cart).map(([id, qty]) => {  
    const p = products.find(x => x.id === id);  
    total += p.price * qty;  
    return { name: p.name, qty, subtotal: p.price * qty };  
  });  
  send(res, 200, { items, total });  
}
```

```
function checkout(req, res) {  
  const cart = carts[req.user.username] || {};  
  const items = Object.entries(cart).map(([id, qty]) => {  
    const p = products.find(x => x.id === id);  
    return { name: p.name, qty, price: p.price };  
  });  
  if (!items.length) return send(res, 400, { error: 'Cart is empty' });  
  const total = items.reduce((t, i) => t + i.price * i.qty, 0);  
  (history[req.user.username] || []).push({ time: new Date().toLocaleString(), items, total });  
  carts[req.user.username] = {};  
  send(res, 200, { message: 'Purchase completed', total });  
}
```

```
const viewHistory = (req, res) => send(res, 200, history[req.user.username] || []);
```

```
function addProduct(req, res) { // admin only
  const { name, price } = req.body;
  if (!name || !price) return send(res, 400, { error: 'name and price required' });
  const p = { id: Math.max(0, ...products.map(x => x.id)) + 1, name, price: Number(price) };
  products.push(p);
  send(res, 201, p);
}
```

```
function deleteProduct(req, res) { // admin only
  const before = products.length;
  products = products.filter(p => p.id !== req.params.id);
  if (products.length === before) return send(res, 404, { error: 'Product not found' });
  send(res, 200, { message: 'Product deleted' });
}
```

```
// ----- ROUTES: [method, path, [middleware..., handler]] -----
```

```
const routes = [
  ['POST', '/api/login', [login]], // public
  ['GET', '/api/profile', [auth, profile]], // protected
  ['GET', '/api/products', [auth, listProducts]], // protected
  ['POST', '/api/cart', [auth, addToCart]], // protected
  ['GET', '/api/cart', [auth, viewCart]], // protected
  ['POST', '/api/checkout', [auth, checkout]], // protected
  ['GET', '/api/history', [auth, viewHistory]], // protected
  ['POST', '/api/products', [auth, requireRole('admin'), addProduct]], // admin only
  ['DELETE', '/api/products/:id', [auth, requireRole('admin'), deleteProduct]], // admin only
];
```

```
// Runs middleware chain like Express's next()
```

```
function runChain(handlers, req, res) {
  let i = 0;
  const next = () => handlers[i++](req, res, next);
  next();
}
```

```
function matchRoute(method, path) {
  for (const [m, p, handlers] of routes) {
    if (m !== method) continue;
    const names = [];
    const regex = new RegExp('^' + p.replace(/:(\w+)/g, (_, n) => (names.push(n), '([^\s]+)')) + '$');
    const match = path.match(regex);
    if (match) {
      const params = {};
      names.forEach((n, idx) => (params[n] = match[idx + 1]));
      return { handlers, params };
    }
  }
  return null;
}
```

// ----- TEST PAGE (frontend to demo the API) -----

```
const PAGE = `<!DOCTYPE html><html><head><meta charset="UTF-8"><title>JWT Demo</title>
<style>
  body{font-family:Arial;max-width:850px;margin:25px auto;padding:0 15px}
  input,button{padding:6px 10px;margin:3px}
  pre{background:#1e1e1e;color:#9cdcfe;padding:12px;border-radius:6px;overflow:auto;min-
    height:120px}
  .box{background:#f4f6f8;padding:10px;border-radius:6px;margin:10px 0;word-break:break-
    all;font-size:13px}
  h3{margin-bottom:4px}
</style></head><body>
<h1>🔒 Shopping API - JWT Demo</h1>

<h3>1. Login (public)</h3>
<input id="user" placeholder="username" value="student">
<input id="pass" placeholder="password" value="1234" type="password">
<button onclick="login()">Login</button>
<small>Users: student/1234 (user), admin/admin123 (admin)</small>
<div class="box"><b>Token:</b> <span id="tok">(not logged in)</span></div>
```

### <h3>2. Protected endpoints (need token)</h3>

```
<button onclick="call('GET','/api/profile')">Profile</button>
<button onclick="call('GET','/api/products')">Products</button>
<input id="pid" type="number" value="1" style="width:60px">
<button onclick="call('POST','/api/cart',{productId:Number(el('pid').value)})">Add to
  Cart</button>
<button onclick="call('GET','/api/cart')">View Cart</button>
<button onclick="call('POST','/api/checkout')">Checkout</button>
<button onclick="call('GET','/api/history')">History</button>
```

### <h3>3. Admin only</h3>

```
<input id="pname" placeholder="name"><input id="pprice" placeholder="price" type="number"
  style="width:80px">
<button
  onclick="call('POST','/api/products',{name:el('pname').value,price:el('pprice').value})">Add
  Product</button>
<input id="did" type="number" value="4" style="width:60px">
<button onclick="call('DELETE','/api/products/'+el('did').value)">Delete Product</button>
```

### <h3>4. Security tests</h3>

```
<button onclick="call('GET','/api/products',null,false)">Call without token</button>
<button onclick="token='abc.def.ghi';call('GET','/api/products')">Call with fake token</button>
<button onclick="logout()">Logout (discard token)</button>
```

<h3>Response</h3><pre id="out">Login first...</pre>

```
<script>
var token = '';
function el(id){ return document.getElementById(id); }
async function call(method, url, body, useToken){
  var h = {'Content-Type':'application/json'};
  if (useToken !== false && token) h['Authorization'] = 'Bearer ' + token;
  var r = await fetch(url, {method: method, headers: h, body: body ? JSON.stringify(body) :
    undefined});
  var d = await r.json();
  el('out').textContent = method + ' ' + url + '\nStatus: ' + r.status + '\n\n' + JSON.stringify(d, null,
    2);
}
```

```

    return d;
}
async function login(){
    var d = await call('POST', '/api/login', {username: el('user').value, password: el('pass').value},
        false);
    if (d.token) { token = d.token; el('tok').textContent = token; }
}
function logout(){ token = ""; el('tok').textContent = '(not logged in)'; el('out').textContent = 'Token
    discarded.'; }
</script></body></html>`;

```

```

// ----- SERVER -----
http.createServer(async (req, res) => {
    try {
        const path = new URL(req.url, 'http://x').pathname;

        if (req.method === 'GET' && path === '/') {
            res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
            return res.end(PAGE);
        }

        const route = matchRoute(req.method, path);
        if (!route) return send(res, 404, { error: 'Route not found' });

        req.params = route.params;
        req.body = ['POST', 'PUT'].includes(req.method) ? await readJSON(req) : {};
        runChain(route.handlers, req, res);
    } catch (e) {
        console.error(e);
        send(res, 500, { error: 'Server error' });
    }
}).listen(3000, () => console.log('Server running at http://localhost:3000'));n(3000, () =>
    console.log('Server running at http://localhost:3000'));

```